

CSP_Analysis_final_v

January 9, 2025

<h1 style="margin-bottom:0.5cm">Final Master Thesis</h1>

<p style="margin-bottom:0.25cm">Course 2024-2025: Machine Learning study using Common Spatial Patterns

Master in Neuroengineering and Rehabilitation</p>

Director Joan F. Alonso

Cootutor Andres El-Fakdi Sencianes

 Student. Ferran Franco

Common Spatial Patterns (CSP) is a signal processing technique used to extract features from EEG data, specifically for distinguishing between two conditions or mental states. CSP works by finding spatial filters that maximize the variance for one class while minimizing it for the other. The filters are derived from the covariance matrices of the EEG signals and are applied to transform the data into a low-dimensional feature space. [1] [2] [3] [4] [5]

How It Works:

Input Data: EEG signals from multiple channels. Covariance Calculation: Compute covariance matrices for each class. Decomposition: Solve a generalized eigenvalue problem to obtain spatial filters. Feature Extraction: Project EEG signals through the filters to create features representing class-specific variance. Selection: Select the most discriminative CSP features, typically the top and bottom few.

Application:

CSP is widely used in Brain-Computer Interface (BCI) systems, particularly for motor imagery tasks. It helps in classifying mental states like imagining left-hand versus right-hand movements, which is crucial for controlling assistive devices like robotic arms or communication systems for individuals with motor impairments.

Strengths:

Effective for binary classification problems with distinct spatial patterns in the EEG. Provides interpretable spatial filters that highlight the brain regions involved in the task.

Limitations:

Sensitive to noise and artifacts; requires pre-processing steps like artifact removal and band-pass filtering. Performance may degrade for multiclass problems without modification or extension (e.g., One-vs-Rest CSP).

Practical Use:

In practice, CSP is often combined with machine learning classifiers (e.g., Support Vector Machines or Logistic Regression) to build robust predictive models for classifying brain states. Its interpretability makes it valuable in neuroscience research for identifying neural correlates of specific tasks or conditions.

```
[1]: import pandas as pd

# Paths to the saved CSP feature files
training_features_path = r"C:\Users\Ferran\Desktop\TFM\TFM_EEG\DATASET\MI\SUB_2\CSP\csp_features_labels.
↳parquet"
validation_features_path = r"C:\Users\Ferran\Desktop\TFM\TFM_EEG\DATASET\MI\SUB_2\CSP\Validation_test\csp_features_labels.
↳parquet"

# Load features
train_df = pd.read_parquet(training_features_path)
val_df = pd.read_parquet(validation_features_path)

print("Training CSP Features:")
display(train_df.head())

print("Validation CSP Features:")
display(val_df.head())
```

Training CSP Features:

	0	1	2	3	labels
0	-2.250265	-3.197191	-2.120102	-1.253858	0
1	-2.474475	-1.047509	-2.057953	-0.780182	0
2	-3.593848	-2.961035	-2.236238	-1.268460	0
3	-3.462869	-2.375898	-0.988663	-1.428627	0
4	-3.141334	-2.768838	-0.713592	-1.142243	0

Validation CSP Features:

	0	1	2	3	labels
0	-3.315264	-1.842860	-3.703404	-3.474552	0
1	-2.021105	-0.901401	-3.422205	-3.905316	0
2	-1.380460	-2.607968	-3.402572	-3.340552	0
3	-3.399149	-2.118043	-3.199128	-3.383165	0
4	-2.264250	-1.154802	-3.310987	-2.780178	0

```
[2]: print("Feature Statistics for Class 0:")
print(train_df[train_df['labels'] == 0].iloc[:, :-1].describe())

print("Feature Statistics for Class 1:")
print(train_df[train_df['labels'] == 1].iloc[:, :-1].describe())
```

Feature Statistics for Class 0:

	0	1	2	3
count	350.000000	350.000000	350.000000	350.000000
mean	-1.646930	-1.907914	-1.552406	-1.208342
std	1.519049	1.030984	1.258406	0.990563
min	-4.851473	-4.383622	-4.598633	-3.816825
25%	-2.739201	-2.605761	-2.409525	-1.832507
50%	-1.978366	-1.958065	-1.798104	-1.317091
75%	-0.711995	-1.319776	-0.864502	-0.662689
max	4.178610	5.477727	4.356457	4.705455

Feature Statistics for Class 1:

	0	1	2	3
count	301.000000	301.000000	301.000000	301.000000
mean	-3.014825	-2.792273	-2.677165	-2.384686
std	0.632123	0.527083	0.552422	0.532631
min	-4.712926	-4.738014	-4.479537	-4.207166
25%	-3.461880	-3.113364	-3.059873	-2.728714
50%	-3.025340	-2.731614	-2.614150	-2.344379
75%	-2.572345	-2.420212	-2.330621	-2.024352
max	-1.183113	-1.615177	-1.340591	-1.082946

```
[ ]: print(train_df.head())
      print(val_df.head())
```

```
[ ]: # Check distributions of CSP features
import matplotlib.pyplot as plt
import seaborn as sns

# Plot distributions of CSP components for training data
for i in range(4): # Assuming 4 CSP components
    sns.histplot(train_df.iloc[:, i], kde=True, label=f"Component {i+1}")
plt.title("Training CSP Feature Distributions")
plt.legend()
plt.show()

# Compare class distributions
sns.countplot(x='labels', data=train_df)
plt.title("Class Distribution in Training Data")
plt.show()
```

```
[ ]: from sklearn.decomposition import PCA
      from sklearn.manifold import TSNE

# Prepare training features and labels
X_train = train_df.iloc[:, :-1].values # Features (exclude the last 'labels'
    ↪ column)
y_train = train_df['labels'].values # Labels
```

```

# PCA visualization
pca = PCA(n_components=2)
pca_result = pca.fit_transform(X_train)

plt.figure(figsize=(8, 6))
sns.scatterplot(x=pca_result[:, 0], y=pca_result[:, 1], hue=y_train,
                palette="Set1")
plt.title("PCA Visualization of CSP Features (Training Data)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.show()

# t-SNE visualization
tsne = TSNE(n_components=2, random_state=42)
tsne_result = tsne.fit_transform(X_train)

plt.figure(figsize=(8, 6))
sns.scatterplot(x=tsne_result[:, 0], y=tsne_result[:, 1], hue=y_train,
                palette="Set1")
plt.title("t-SNE Visualization of CSP Features (Training Data)")
plt.xlabel("t-SNE Dimension 1")
plt.ylabel("t-SNE Dimension 2")
plt.show()

```

```

[ ]: import joblib
import matplotlib.pyplot as plt
from mne import create_info
from mne.channels import make_standard_montage
from mne.viz import plot_topomap

# Load the CSP model
csp_model_path = "C:
↳\Users\Ferran\Desktop\TFM\TFM_EEG\DATASET\MI\SUB_2\CSP\csp_model.pkl"
csp = joblib.load(csp_model_path)

# Set channel info (must match training configuration)
ch_names = ['F3', 'Fz', 'F4', 'C3', 'Cz', 'C4', 'P3', 'Pz', 'P4']
sfreq = 250 # Sampling frequency used during training
info = create_info(ch_names=ch_names, sfreq=sfreq, ch_types='eeg')

# Attach a standard 10-20 montage to provide electrode positions
montage = make_standard_montage('standard_1020')
info.set_montage(montage)

# Select patterns corresponding to the 10 channels used during training
patterns_to_plot = csp.patterns_[:len(ch_names), :]

```

```

# Plot spatial patterns
fig, axes = plt.subplots(1, 4, figsize=(15, 5)) # Adjust for the number of
↳ components (e.g., 4)
for i, ax in enumerate(axes):
    plot_topomap(patterns_to_plot[:, i], info, axes=ax, show=False)
    ax.set_title(f"CSP Pattern {i+1}")
plt.tight_layout()
plt.show()

```

```

[ ]: import seaborn as sns
import matplotlib.pyplot as plt

# Define colors for training and validation components
colors = ['blue', 'green', 'orange', 'red'] # Adjust colors as needed

# Compare distributions of CSP components between training and validation
plt.figure(figsize=(10, 6))
for i in range(4): # Assuming 4 components
    sns.kdeplot(
        train_df.iloc[:, i],
        color=colors[i],
        label=f"Training Component {i+1}",
        linestyle='-'
    )
    sns.kdeplot(
        val_df.iloc[:, i],
        color=colors[i],
        label=f"Validation Component {i+1}",
        linestyle='--'
    )
plt.title("Comparison of CSP Feature Distributions (Training vs Validation)",
↳ fontsize=14)
plt.xlabel("CSP Component Values", fontsize=12)
plt.ylabel("Density", fontsize=12)
plt.legend(title="Legend", fontsize=10)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

```

[4]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, AgglomerativeClustering, DBSCAN, OPTICS
from sklearn.cluster import MeanShift, estimate_bandwidth
from sklearn.mixture import GaussianMixture

```

```

from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.metrics import davies_bouldin_score, adjusted_mutual_info_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
from yellowbrick.cluster import KElbowVisualizer, SilhouetteVisualizer
from pathlib import Path
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, \
    ↪roc_auc_score, classification_report, confusion_matrix, \
    ↪ConfusionMatrixDisplay
import warnings
warnings.filterwarnings('ignore')
from sklearn.preprocessing import StandardScaler

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, \
    ↪roc_auc_score

import joblib

```

```

[5]: file_path = r"C:
    ↪\Users\Ferran\Desktop\TFM\TFM_EEG\DATASET\MI\SUB_2\CSP\csp_features_labels.
    ↪parquet"
# file_path = r"C:
    ↪\Users\Ferran\Desktop\TFM\TFM_EEG\DATASET\MI\SUB_2\CSP\Test_v2\csp_features_labels.
    ↪parquet"
# Load the Parquet file
data = pd.read_parquet(file_path)

# Display the first few rows of the DataFrame
data

```

```

[5]:

```

	0	1	2	3	labels
0	-2.250265	-3.197191	-2.120102	-1.253858	0
1	-2.474475	-1.047509	-2.057953	-0.780182	0
2	-3.593848	-2.961035	-2.236238	-1.268460	0
3	-3.462869	-2.375898	-0.988663	-1.428627	0
4	-3.141334	-2.768838	-0.713592	-1.142243	0
..	...	...	...	...	...
646	-2.524649	-2.303035	-2.765983	-3.459513	1
647	-2.341928	-3.077011	-3.032143	-1.992895	1
648	-1.823746	-2.565983	-2.533202	-1.993773	1
649	-2.538709	-3.674447	-2.235387	-2.075693	1
650	-3.680346	-2.949249	-2.439276	-2.226973	1

[651 rows x 5 columns]

```
[6]: # ADDED CODE WHEN WORKING WITH 4 FEATURES

# Combine features into a single column
data['features'] = data[['0', '1', '2', '3']].values.tolist()

# Drop the individual feature columns
data.drop(columns=['0', '1', '2', '3'], inplace=True)

# Reorder columns so 'features' comes before 'labels'
data = data[['features', 'labels']]

# Check the resulting DataFrame
print(data.head())
```

	features	labels
0	[-2.250265318832594, -3.1971913387863973, -2.1...	0
1	[-2.4744754938027644, -1.047509196086504, -2.0...	0
2	[-3.5938479311874705, -2.961035415128572, -2.2...	0
3	[-3.4628689450534704, -2.3758979635543964, -0...	0
4	[-3.1413344337031557, -2.768838148513806, -0.7...	0

```
[7]: # Flatten and handle inconsistent lengths
# Assume the features are 2D or nested; we need to flatten them
X = [np.ravel(feature) for feature in data['features']]

# Find the maximum length of the flattened features
max_length = max(len(x) for x in X)

# Pad all feature arrays to the maximum length
X_padded = np.array([np.pad(x, (0, max_length - len(x)), mode='constant') for x
    in X])

# Extract labels
y = data['labels']
```

```
[8]: # Flatten each nested array within X_padded
X_flattened = np.array([np.hstack(sample) for sample in X_padded])

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_flattened)

print("Normalized features shape:", X_scaled.shape)
```

Normalized features shape: (651, 4)

```
[9]: print("Shape of X_flattened:", X_flattened.shape)
      print("Example flattened element:", X_flattened[0])

      print("Shape of X_scaled:", X_scaled.shape)
      print("Example normalized element:", X_scaled[0])
```

Shape of X_flattened: (651, 4)
 Example flattened element: [-2.25026532 -3.19719134 -2.12010247 -1.25385762]
 Shape of X_scaled: (651, 4)
 Example normalized element: [0.021211 -0.93201059 -0.04172383 0.49819839]

```
[10]: # Split the data
      X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.4,
      ↪random_state=42)
```

```
[11]: # Initialize the KNN model with 3 neighbors
      knn = KNeighborsClassifier(n_neighbors=3)

      # Train the model
      knn.fit(X_train, y_train)

      # Predict on the test set
      y_pred = knn.predict(X_test)
      y_pred_proba = knn.predict_proba(X_test)[: , 1]

      # Evaluate the model
      accuracy = accuracy_score(y_test, y_pred)
      precision = precision_score(y_test, y_pred)
      recall = recall_score(y_test, y_pred)
      roc_auc = roc_auc_score(y_test, y_pred_proba)

      # Print evaluation metrics
      print("KNN Evaluation Metrics:")
      print(f"Accuracy: {accuracy}")
      print(f"Precision: {precision}")
      print(f"Recall: {recall}")
      print(f"ROC AUC: {roc_auc}")
```

KNN Evaluation Metrics:
 Accuracy: 0.89272030651341
 Precision: 0.8181818181818182
 Recall: 0.9428571428571428
 ROC AUC: 0.9443223443223443

```
[12]: # Load the Parquet file
      # validation_path = r"C:
      ↪\Users\Ferran\Desktop\TFM\TFM_EEG\DATASET\MI\SUB_2\CSP\Validation\csp_features_labels_valid
      ↪parquet"
```



```
validation_path = r"C:
↳\Users\Ferran\Desktop\TFM\TFM_EEG\DATASET\MI\SUB_2\CSP\Validation_test\csp_features_labels_
↳parquet"
validation_data = pd.read_parquet(validation_path)

# Display the first few rows of the DataFrame
validation_data
```

```
[12]:
```

	0	1	2	3	labels
0	-3.315264	-1.842860	-3.703404	-3.474552	0
1	-2.021105	-0.901401	-3.422205	-3.905316	0
2	-1.380460	-2.607968	-3.402572	-3.340552	0
3	-3.399149	-2.118043	-3.199128	-3.383165	0
4	-2.264250	-1.154802	-3.310987	-2.780178	0
..	...	...	...	...	...
235	-3.554519	-3.159049	-1.771760	-1.575974	1
236	-4.398436	-4.478777	-0.555702	-1.300987	1
237	-3.981576	-3.329080	-0.926834	-0.580407	1
238	-4.450248	-2.938428	-2.465475	-2.770043	1
239	-4.292183	-3.501134	-2.171373	-2.463118	1

[240 rows x 5 columns]

```
[13]: # Count the occurrences of each value in the 'side' column
side_counts = validation_data['labels'].value_counts()

print("Counts of rows for each side:")
print(side_counts)
```

Counts of rows for each side:

labels

0 120

1 120

Name: count, dtype: int64

There are the same number of instances of right and left, this means that the validation dataset is equilibrated

```
[14]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
```

```

    ConfusionMatrixDisplay,
    roc_curve,
)
import seaborn as sns

def evaluate_model(model, X, y, label_encoder=None, model_name="Model"):
    """
    Evaluates and visualizes the performance of a classification model.

    Parameters:
    - model: A pre-trained scikit-learn compatible classification model.
    - X: Preprocessed feature matrix (e.g., scaled and imputed).
    - y: Target vector (Pandas Series or NumPy array).
    - label_encoder: An optional LabelEncoder object for decoding labels.
    ↳ Required for visualization.
    - model_name: A string representing the name of the model (e.g., "KNN",
    ↳ "Decision Tree").

    Returns:
    - metrics: A dictionary containing evaluation metrics (accuracy, precision,
    ↳ recall, roc_auc, fpr, tpr).
    """
    # Predict on the dataset
    y_pred = model.predict(X)
    y_pred_proba = (
        model.predict_proba(X)[:, 1]
        if hasattr(model, "predict_proba")
        else model.decision_function(X)
        if hasattr(model, "decision_function")
        else None
    )

    # Calculate metrics
    accuracy = accuracy_score(y, y_pred)
    precision = precision_score(y, y_pred, zero_division=0)
    recall = recall_score(y, y_pred, zero_division=0)
    roc_auc = roc_auc_score(y, y_pred_proba) if y_pred_proba is not None else
    ↳ None

    # Calculate FPR and TPR for the ROC curve (if applicable)
    fpr, tpr, thresholds = (roc_curve(y, y_pred_proba) if y_pred_proba is not
    ↳ None else (None, None, None))

    metrics = {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,

```

```

        "roc_auc": roc_auc,
        "fpr": fpr,
        "tpr": tpr,
    }

    # Print metrics
    print(f"\n{model_name} Evaluation Metrics:")
    for key, value in metrics.items():
        if key in ["fpr", "tpr"]: # Skip printing FPR and TPR as they are
↪arrays
            continue
        print(f"{key.capitalize()}: {value:.4f}" if value is not None else
↪f"{key.capitalize()}: N/A")

    # Confusion Matrix
    cm = confusion_matrix(y, y_pred)
    cm_display = ConfusionMatrixDisplay(confusion_matrix=cm,
↪display_labels=label_encoder.classes_ if label_encoder else None)
    cm_display.plot(cmap="Blues")
    plt.title(f"{model_name} Confusion Matrix")
    plt.show()

    # Classification Report Heatmap
    report = classification_report(y, y_pred, output_dict=True)
    report_df = pd.DataFrame(report).transpose()

    plt.figure(figsize=(10, 6))
    sns.heatmap(report_df.iloc[:-1, :-1], annot=True, cmap="YlGn", fmt=".2f",
↪cbar=True)
    plt.title(f"{model_name} Classification Report")
    plt.xticks(rotation=45)
    plt.yticks(rotation=0)
    plt.show()

    # ROC Curve (if applicable)
    if y_pred_proba is not None:
        plt.figure(figsize=(8, 6))
        plt.plot(fpr, tpr, label=f"ROC Curve (AUC = {roc_auc:.4f})")
        plt.plot([0, 1], [0, 1], linestyle="--", color="red")
        plt.xlabel("False Positive Rate")
        plt.ylabel("True Positive Rate")
        plt.title(f"{model_name} ROC Curve")
        plt.legend(loc="best")
        plt.grid()
        plt.show()

    return metrics

```

```
[15]: # ADDED CODE WHEN WORKING WITH 4 FEATURES

# Combine features into a single column
validation_data['features'] = validation_data[['0', '1', '2', '3']].values.
    ↪tolist()

# Drop the individual feature columns
validation_data.drop(columns=['0', '1', '2', '3'], inplace=True)

# Reorder columns so 'features' comes before 'labels'
validation_data = validation_data[['features', 'labels']]

# Check the resulting DataFrame
print(validation_data.head())
```

	features	labels
0	[-3.315264148474101, -1.842860248827048, -3.70...	0
1	[-2.0211048268475746, -0.9014008513823994, -3...	0
2	[-1.3804598301554856, -2.6079676931962537, -3...	0
3	[-3.399149459984842, -2.1180429687715137, -3.1...	0
4	[-2.2642497607341037, -1.154802385828275, -3.3...	0

```
[16]: # Flatten and handle inconsistent lengths
# Assume the features are 2D or nested; we need to flatten them
X_val = [np.ravel(feature) for feature in validation_data['features']]

# Find the maximum length of the flattened features
max_length_val = max(len(x) for x in X_val)

# Pad all feature arrays to the maximum length
X_padded_val = np.array([np.pad(x, (0, max_length_val - len(x)),
    ↪mode='constant') for x in X_val])

# Extract labels and encode
y = validation_data['labels']
label_encoder = LabelEncoder()
y_validation_encoded = label_encoder.fit_transform(y)

# Flatten each nested array within X_padded
X_flattened_val = np.array([np.hstack(sample) for sample in X_padded_val])

# Use the scaler from training to transform validation data
X_scaled_val = scaler.transform(X_flattened_val)
print("Normalized features shape:", X_scaled_val.shape)
```

Normalized features shape: (240, 4)

0.1 DECISION TREE

```
[17]: # Train a Decision Tree Classifier
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

# Predict and evaluate
y_pred = dt.predict(X_scaled_val)
y_pred_proba = dt.predict_proba(X_scaled_val)[:, 1]

accuracy = accuracy_score(y_validation_encoded, y_pred)
precision = precision_score(y_validation_encoded, y_pred)
recall = recall_score(y_validation_encoded, y_pred)
roc_auc = roc_auc_score(y_validation_encoded, y_pred_proba)

dt_metrics = evaluate_model(dt, X_scaled_val, y_validation_encoded,
                             label_encoder, model_name="Decision Tree")
joblib.dump(dt, 'trained_model_DT_CSP.pkl')

accuracy_dt = dt_metrics["accuracy"]
precision_dt = dt_metrics["precision"]
recall_dt = dt_metrics["recall"]
roc_auc_dt = dt_metrics["roc_auc"]
fpr_dt, tpr_dt = dt_metrics["fpr"], dt_metrics["tpr"]
```

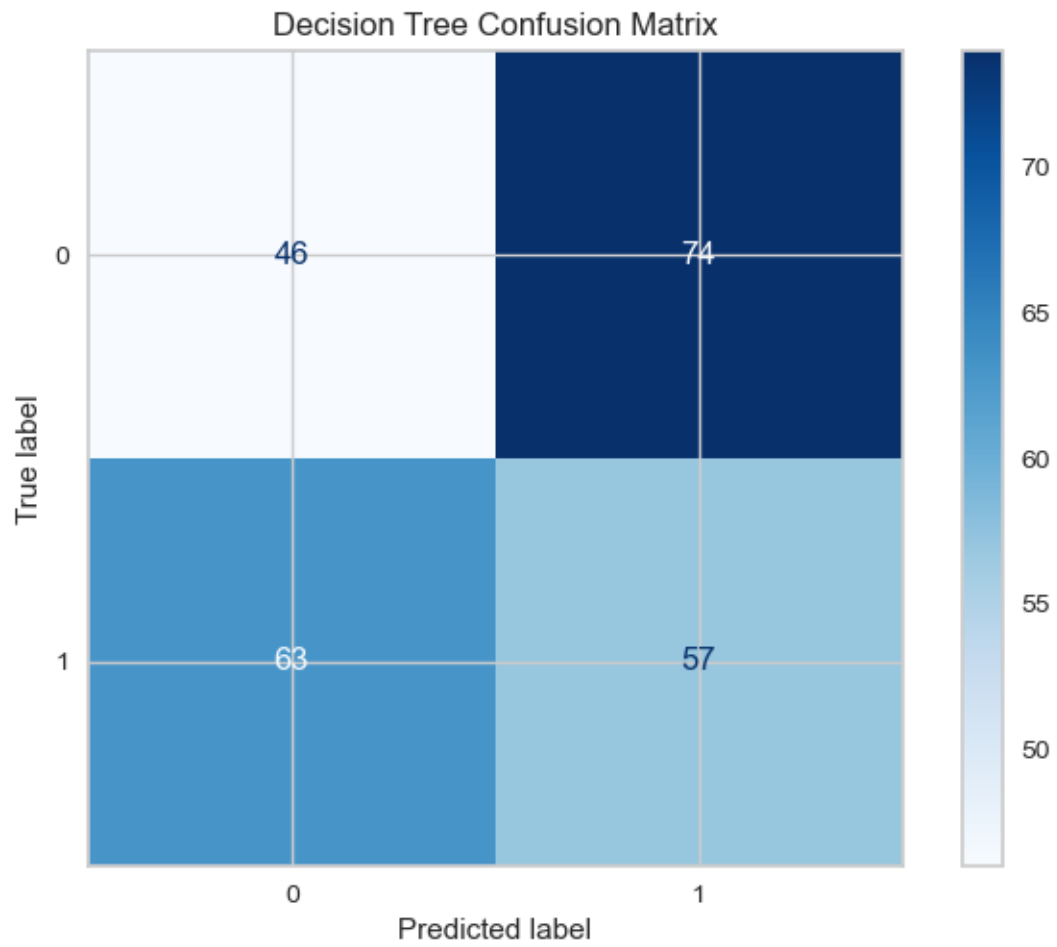
Decision Tree Evaluation Metrics:

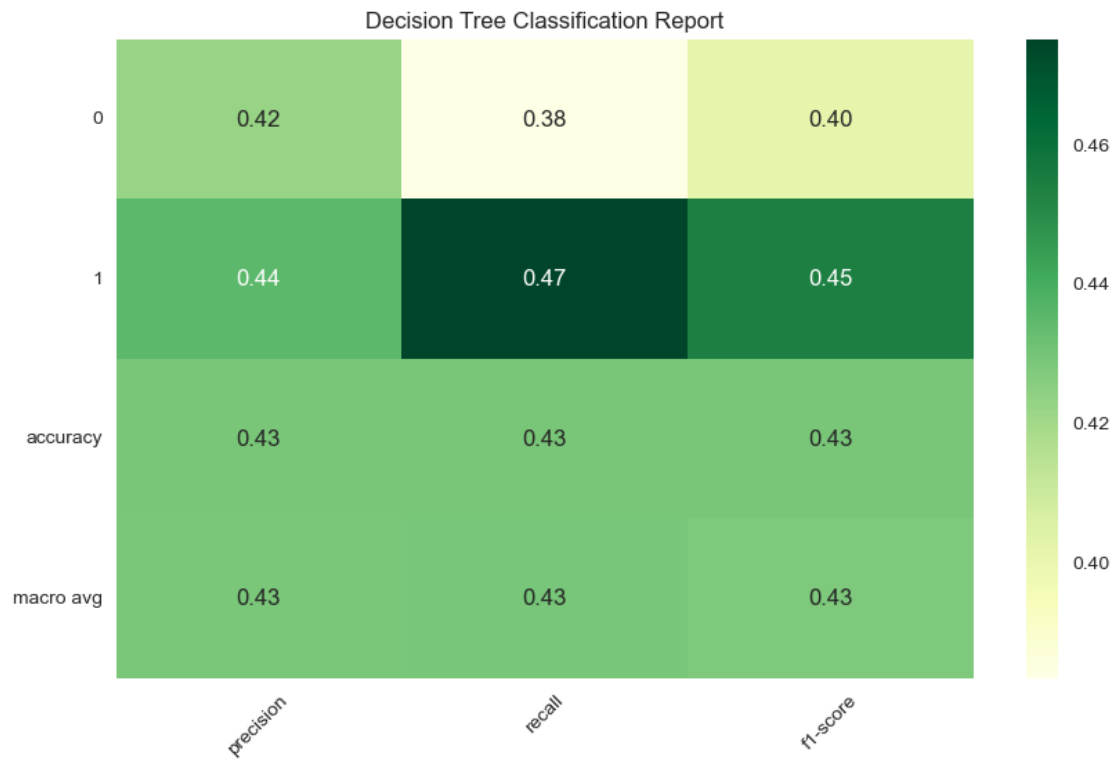
Accuracy: 0.4292

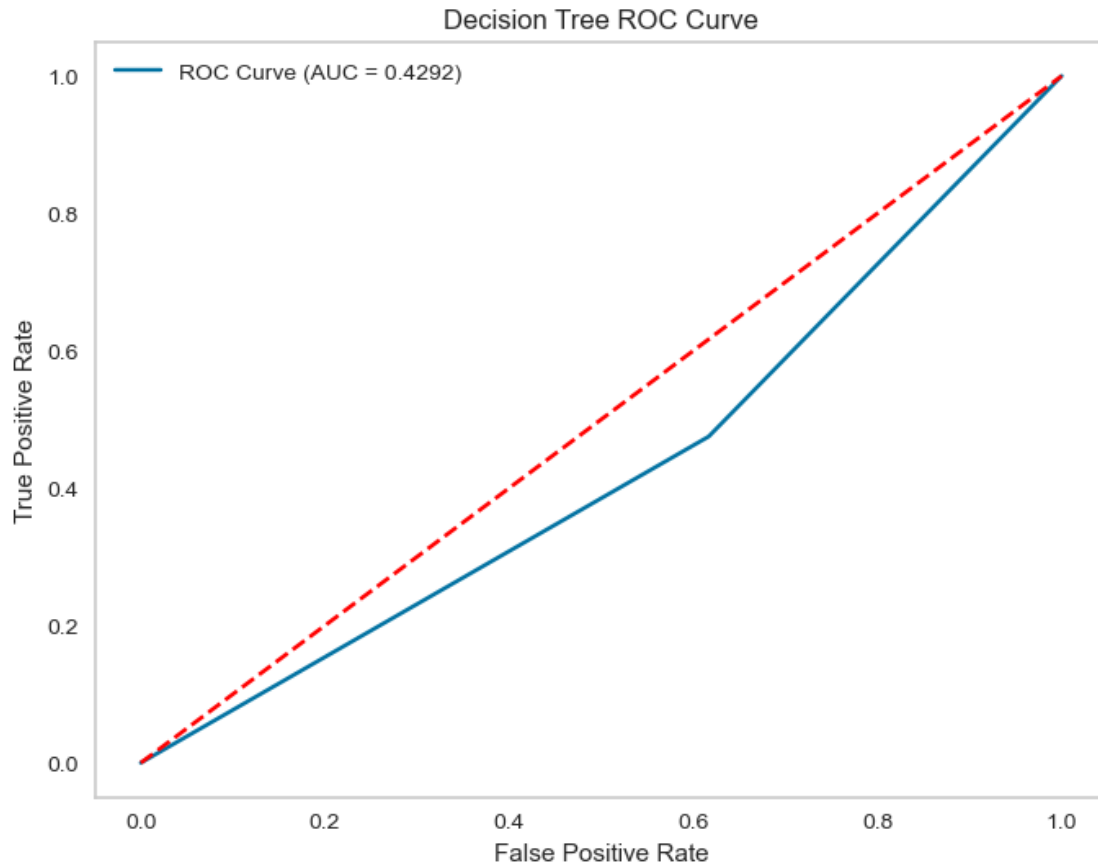
Precision: 0.4351

Recall: 0.4750

Roc_auc: 0.4292







0.2 KNN

```
[18]: # Step 2: Predict on the validation set
y_pred_knn = knn.predict(X_scaled_val)
y_pred_proba_knn = knn.predict_proba(X_scaled_val)[: , 1] # Probability of
↳class 1

# Step 3: Evaluate the KNN model using validation data
accuracy_knn = accuracy_score(y_validation_encoded, y_pred_knn)
precision_knn = precision_score(y_validation_encoded, y_pred_knn)
recall_knn = recall_score(y_validation_encoded, y_pred_knn)
roc_auc_knn = roc_auc_score(y_validation_encoded, y_pred_proba_knn)

# Step 4: Visualize and summarize performance
print("KNN Evaluation:")
knn_metrics = evaluate_model(knn, X_scaled_val, y_validation_encoded,
↳label_encoder, model_name="KNN")
joblib.dump(knn, 'trained_model_KNN_CSP.pkl')
```



```
accuracy_knn = knn_metrics["accuracy"]
precision_knn = knn_metrics["precision"]
recall_knn = knn_metrics["recall"]
roc_auc_knn = knn_metrics["roc_auc"]
fpr_knn, tpr_knn = knn_metrics["fpr"], knn_metrics["tpr"]
```

KNN Evaluation:

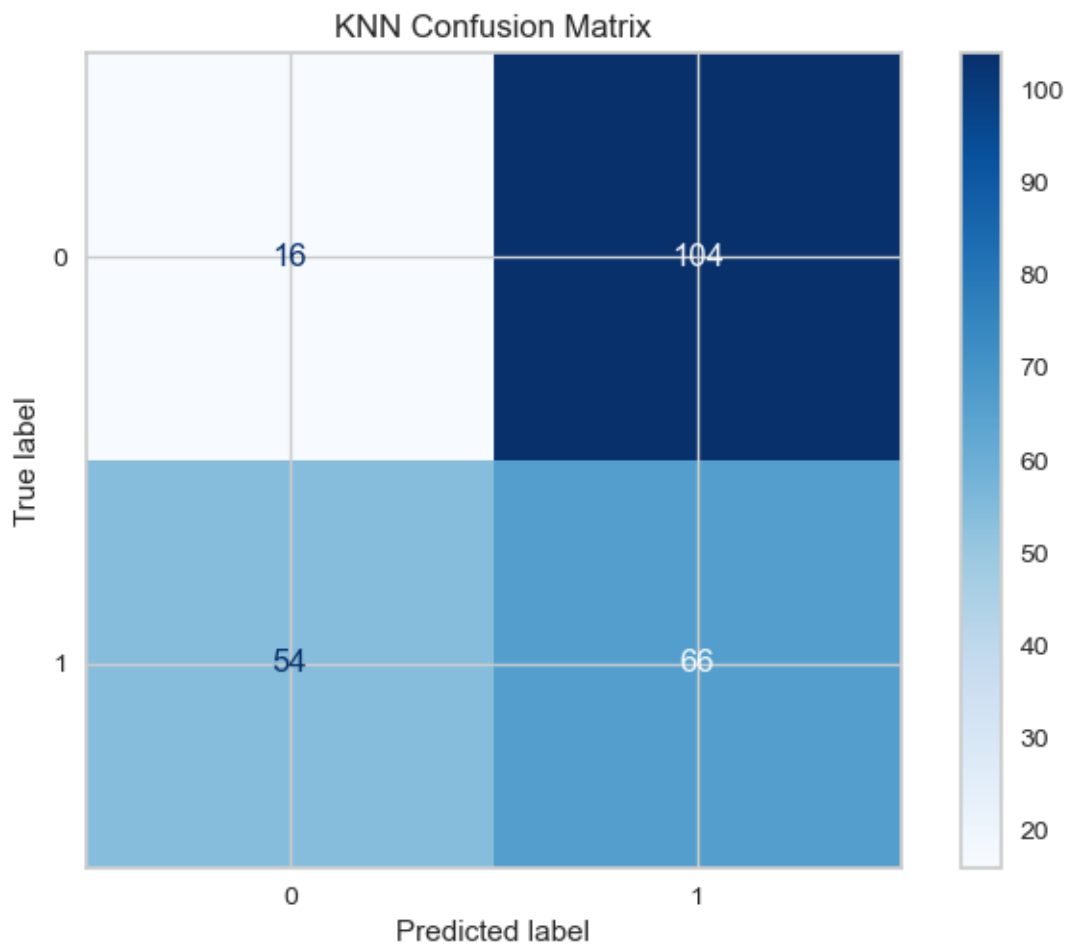
KNN Evaluation Metrics:

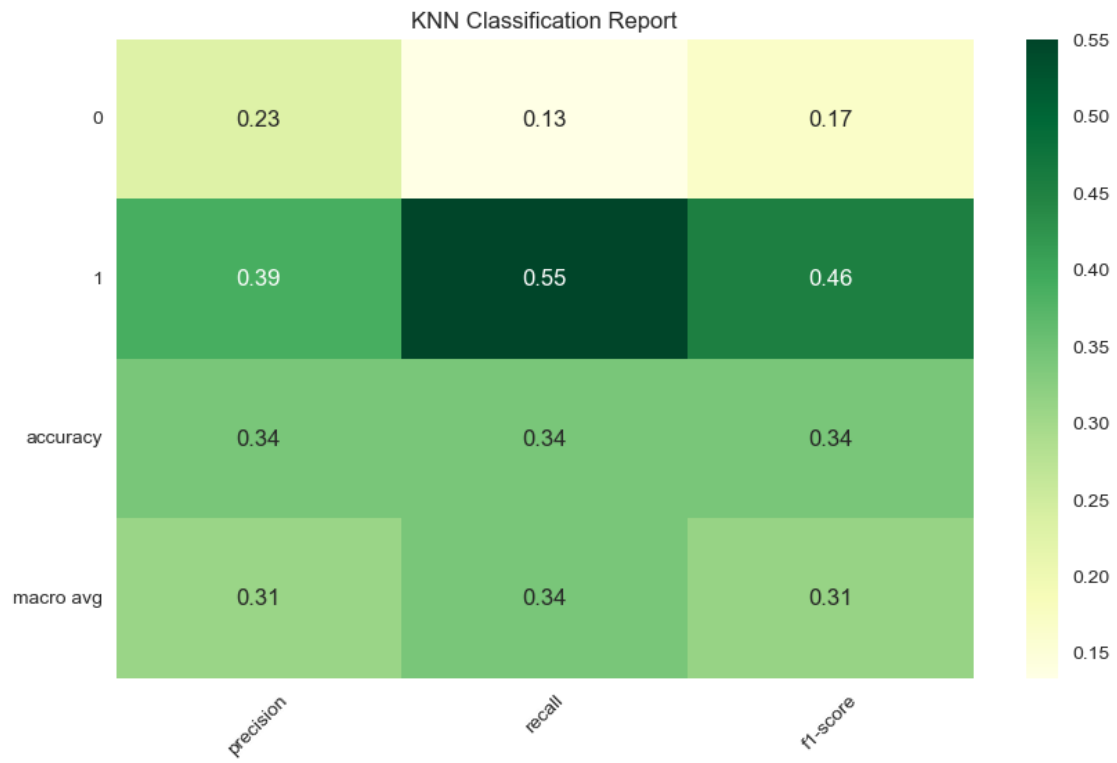
Accuracy: 0.3417

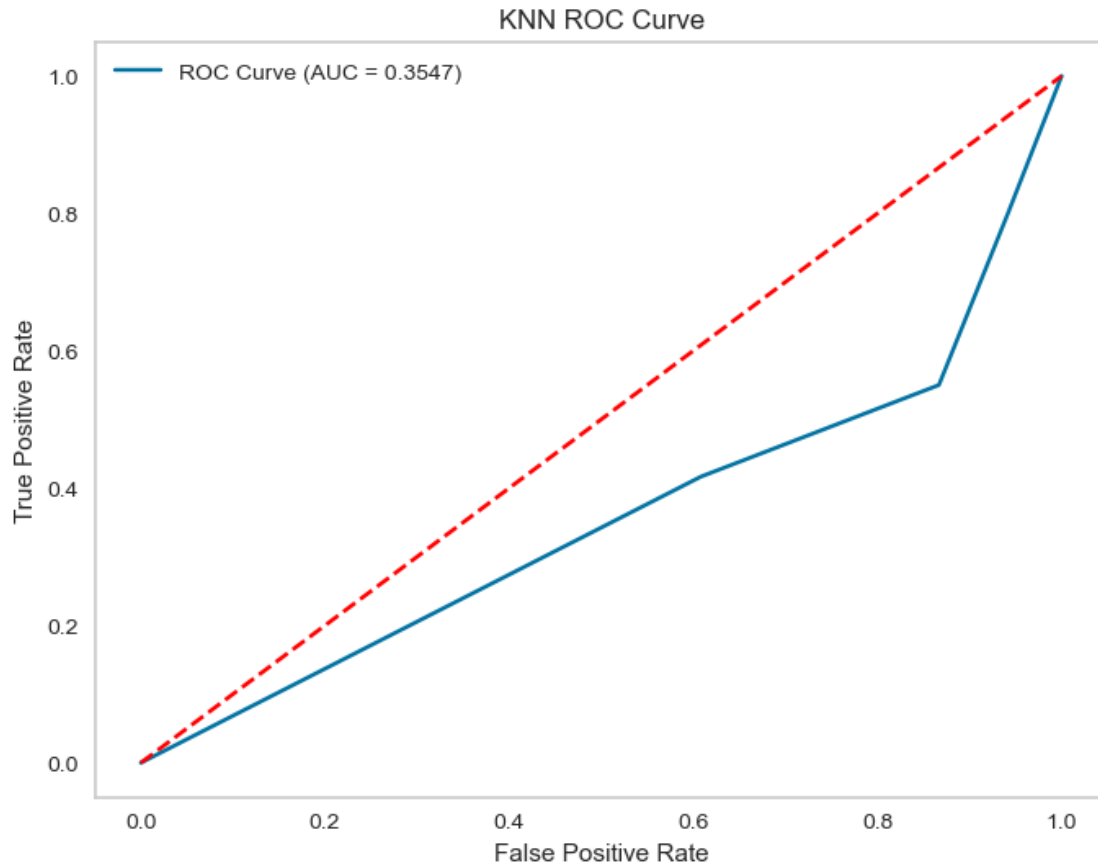
Precision: 0.3882

Recall: 0.5500

Roc_auc: 0.3547







0.3 SVM

```
[19]: # Step 1: Initialize the SVM model
svm = SVC(kernel='linear', C=1.0, probability=True, class_weight='balanced') #
    ↳ Linear kernel, balanced class weights

# Step 2: Train the SVM model on the training data
svm.fit(X_train, y_train)

# Step 3: Predict on the validation set
y_pred_svm = svm.predict(X_scaled_val)
y_pred_proba_svm = svm.predict_proba(X_scaled_val)[: , 1] # Probability of
    ↳ class 1

# Step 4: Evaluate the SVM model using validation data
accuracy_svm = accuracy_score(y_validation_encoded, y_pred_svm)
precision_svm = precision_score(y_validation_encoded, y_pred_svm)
recall_svm = recall_score(y_validation_encoded, y_pred_svm)
roc_auc_svm = roc_auc_score(y_validation_encoded, y_pred_proba_svm)
```

```
# Step 5: Visualize and summarize performance
print("SVM Evaluation:")
svm_metrics = evaluate_model(svm, X_scaled_val, y_validation_encoded,
                             label_encoder, model_name="SVM")
joblib.dump(svm, 'trained_model_SVM_CSP.pkl')

accuracy_svm = svm_metrics["accuracy"]
precision_svm = svm_metrics["precision"]
recall_svm = svm_metrics["recall"]
roc_auc_svm = svm_metrics["roc_auc"]
fpr_svm, tpr_svm = svm_metrics["fpr"], svm_metrics["tpr"]
```

SVM Evaluation:

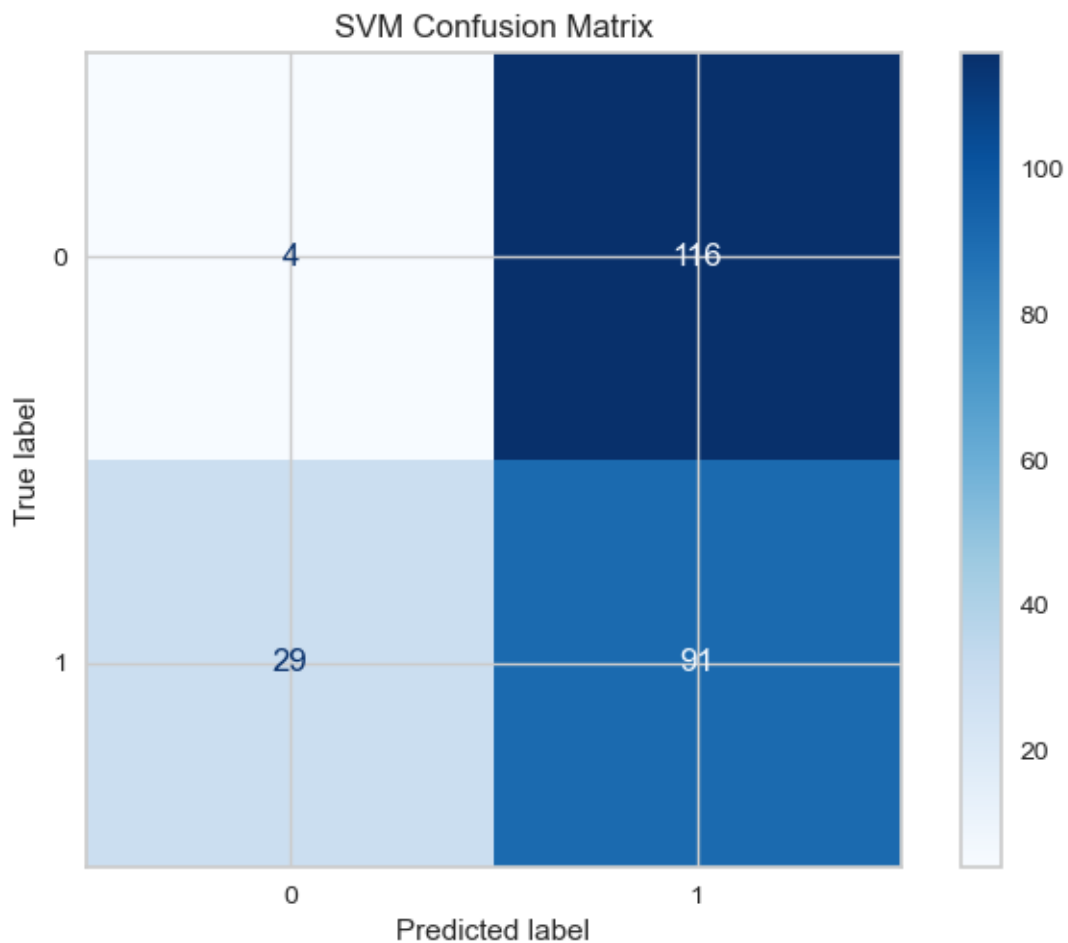
SVM Evaluation Metrics:

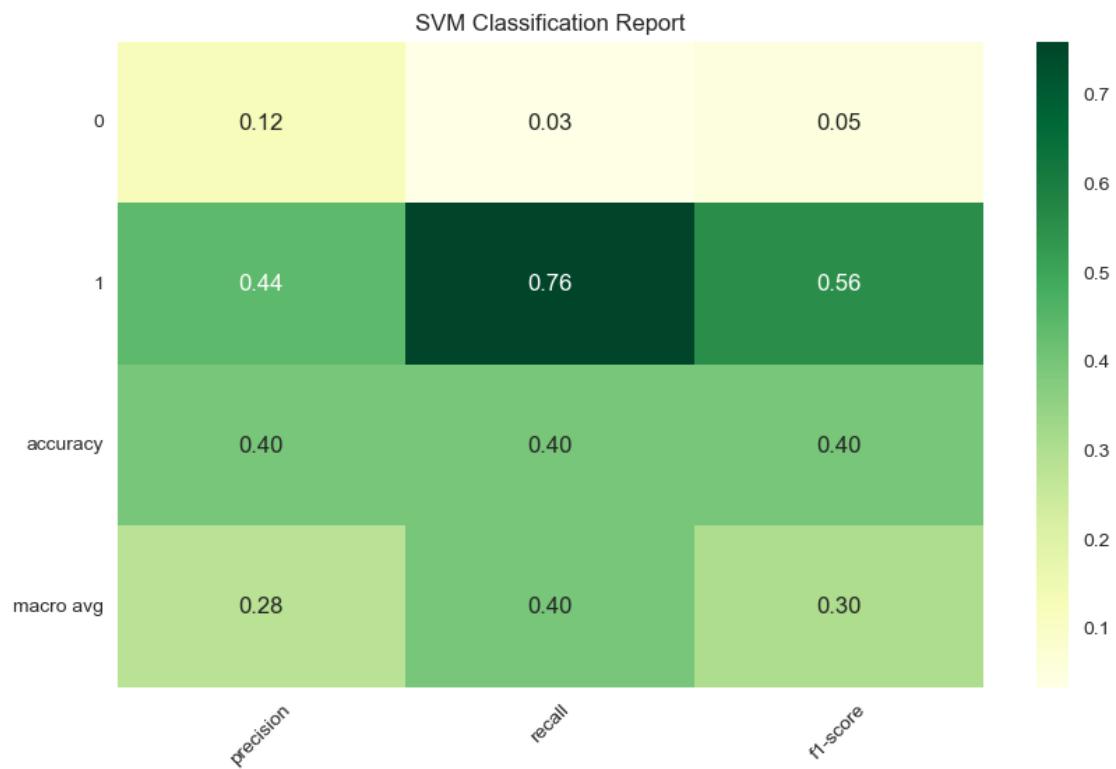
Accuracy: 0.3958

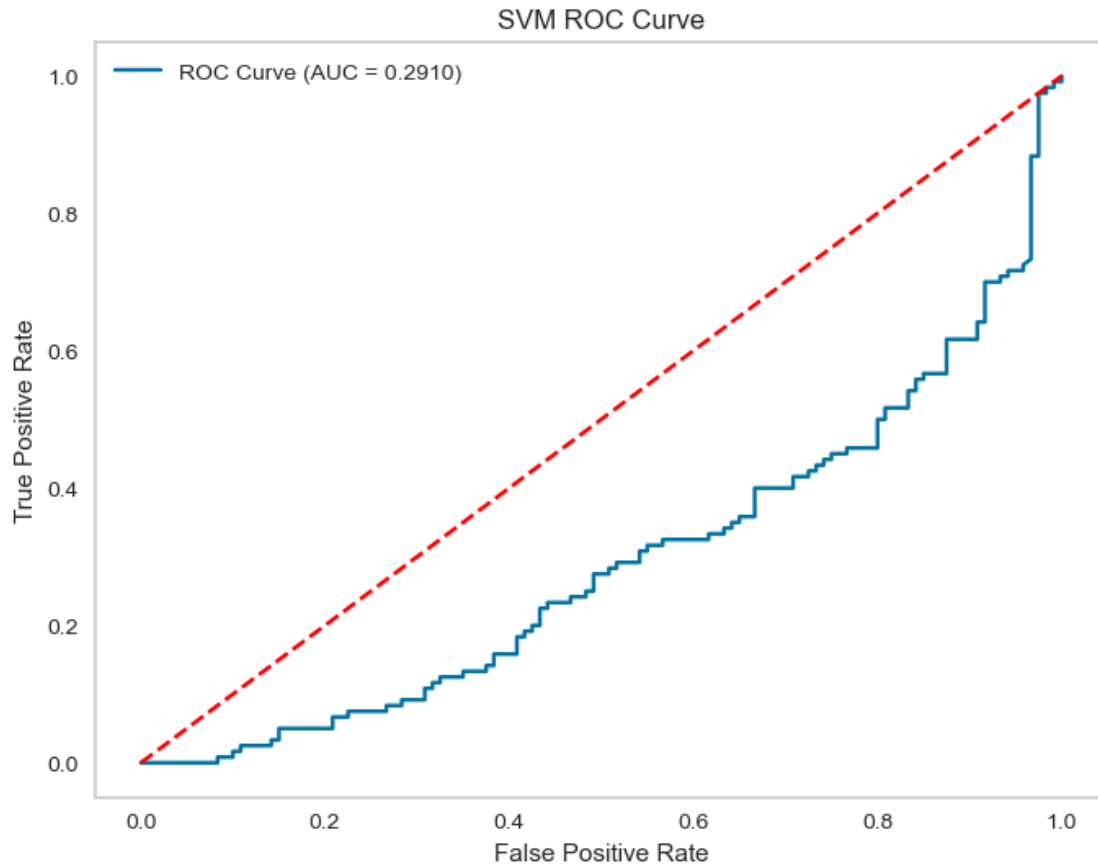
Precision: 0.4396

Recall: 0.7583

Roc_auc: 0.2910







0.4 RANDOM FOREST

```
[20]: # Step 1: Initialize the Random Forest model
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, class_weight='balanced',
    ↪random_state=42) # Balanced class weights

# Step 2: Train the Random Forest model on the training data
rf.fit(X_train, y_train)

# Step 3: Predict on the validation set
y_pred_rf = rf.predict(X_scaled_val)
y_pred_proba_rf = rf.predict_proba(X_scaled_val)[: , 1] # Probability of class 1

# Step 4: Evaluate the Random Forest model using validation data
accuracy_rf = accuracy_score(y_validation_encoded, y_pred_rf)
precision_rf = precision_score(y_validation_encoded, y_pred_rf)
recall_rf = recall_score(y_validation_encoded, y_pred_rf)
```

```

roc_auc_rf = roc_auc_score(y_validation_encoded, y_pred_proba_rf)

# Step 5: Visualize and summarize performance
print("Random Forest Evaluation:")
rf_metrics = evaluate_model(rf, X_scaled_val, y_validation_encoded,
    ↪label_encoder, model_name="Random Forest")

joblib.dump(rf, 'trained_model_RF_CSP.pkl')

accuracy_rf = rf_metrics["accuracy"]
precision_rf = rf_metrics["precision"]
recall_rf = rf_metrics["recall"]
roc_auc_rf = rf_metrics["roc_auc"]
fpr_rf, tpr_rf = rf_metrics["fpr"], rf_metrics["tpr"]

```

Random Forest Evaluation:

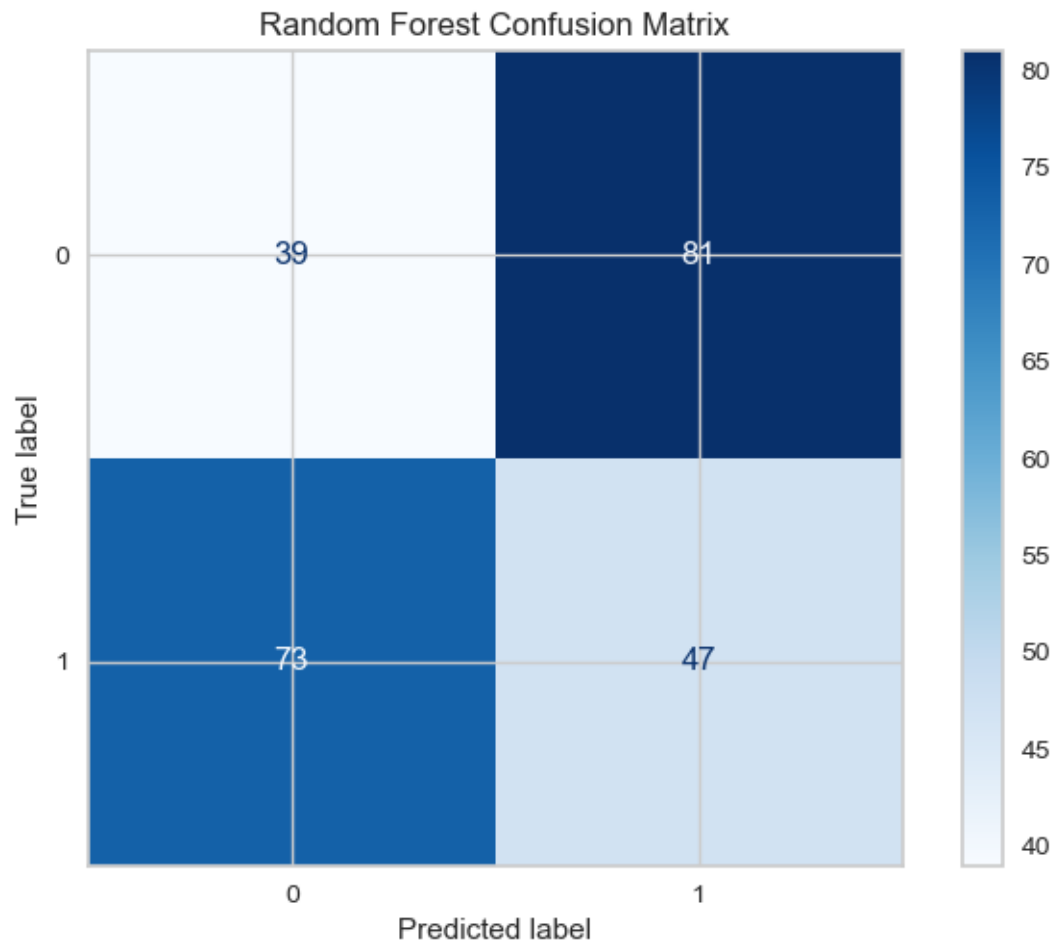
Random Forest Evaluation Metrics:

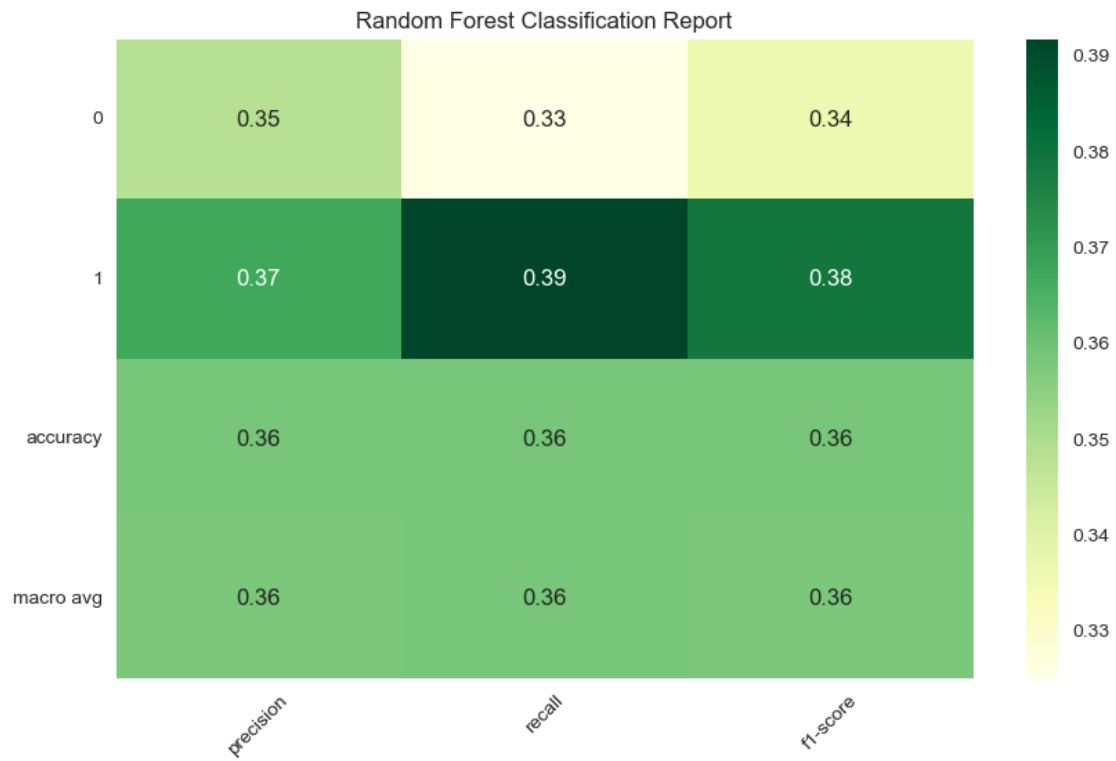
Accuracy: 0.3583

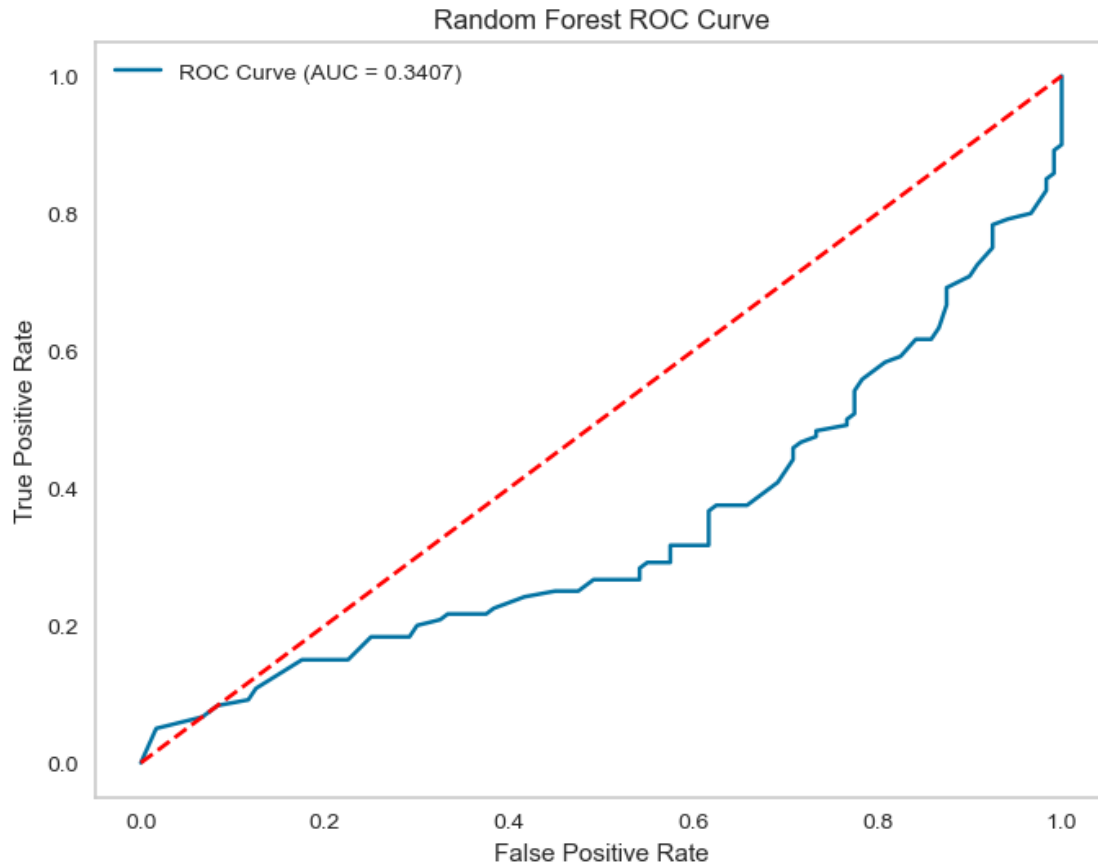
Precision: 0.3672

Recall: 0.3917

Roc_auc: 0.3407







0.5 LOGISTIC REGRESSION

```
[21]: # Step 1: Initialize the Logistic Regression model
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(class_weight='balanced', random_state=42,
    ↪ solver='liblinear') # Balanced class weights

# Step 2: Train the Logistic Regression model on the training data
lr.fit(X_train, y_train)

# Step 3: Predict on the validation set
y_pred_lr = lr.predict(X_scaled_val)
y_pred_proba_lr = lr.predict_proba(X_scaled_val)[: , 1] # Probability of class 1

# Step 4: Evaluate the Logistic Regression model using validation data
accuracy_lr = accuracy_score(y_validation_encoded, y_pred_lr)
precision_lr = precision_score(y_validation_encoded, y_pred_lr)
recall_lr = recall_score(y_validation_encoded, y_pred_lr)
```

```

roc_auc_lr = roc_auc_score(y_validation_encoded, y_pred_proba_lr)

# Step 5: Visualize and summarize performance
print("Logistic Regression Evaluation:")
lr_metrics = evaluate_model(lr, X_scaled_val, y_validation_encoded,
                             label_encoder, model_name="Logistic Regression")
joblib.dump(lr, 'trained_model_LR_CSP.pkl')

accuracy_lr = lr_metrics["accuracy"]
precision_lr = lr_metrics["precision"]
recall_lr = lr_metrics["recall"]
roc_auc_lr = lr_metrics["roc_auc"]
fpr_lr, tpr_lr = lr_metrics["fpr"], lr_metrics["tpr"]

```

Logistic Regression Evaluation:

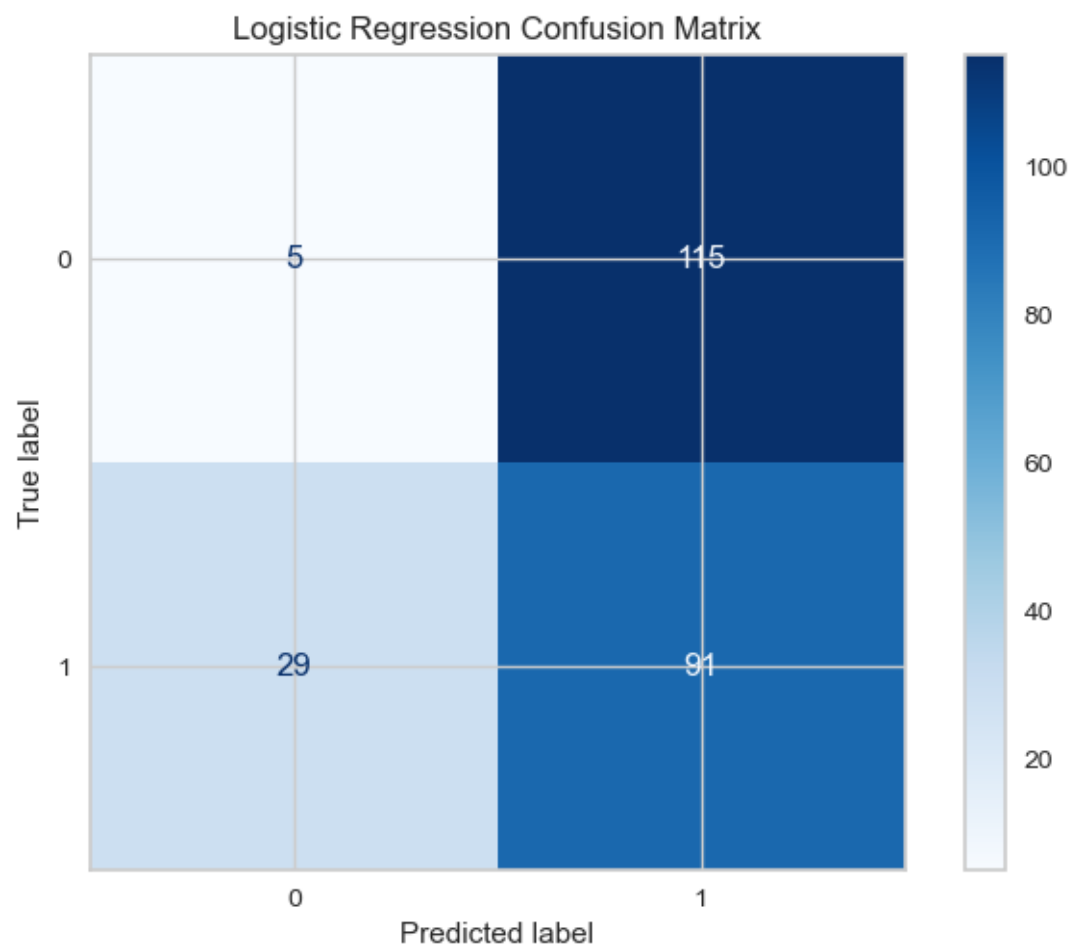
Logistic Regression Evaluation Metrics:

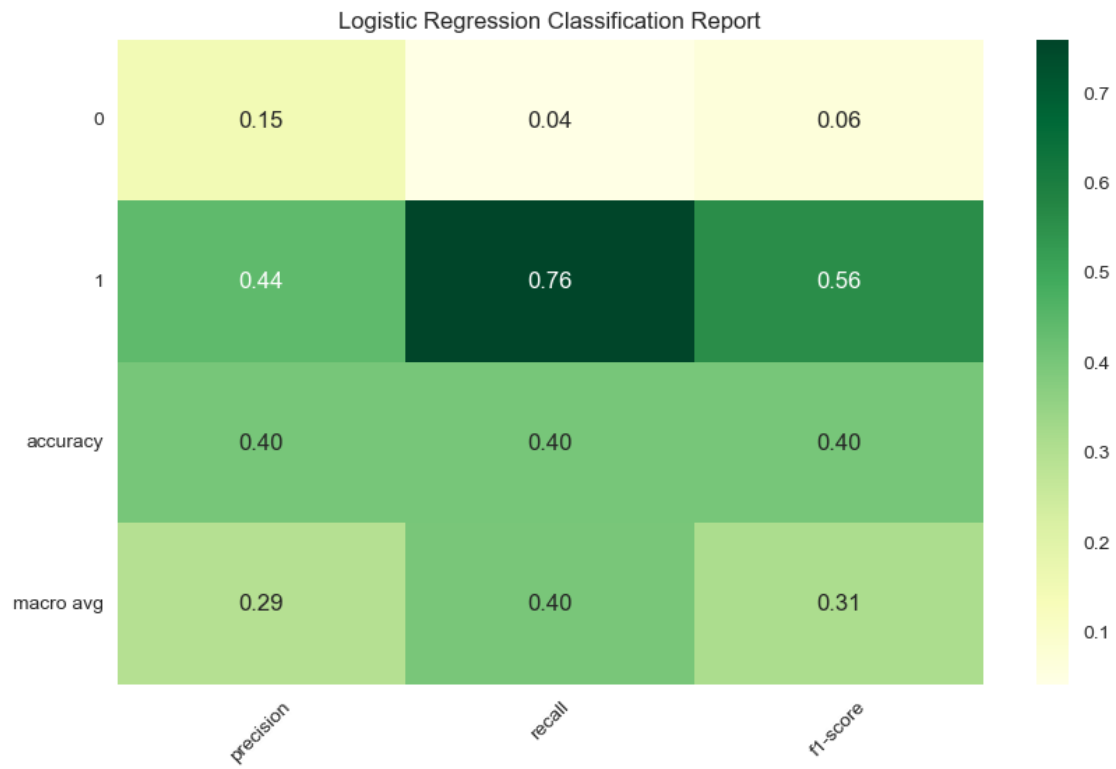
Accuracy: 0.4000

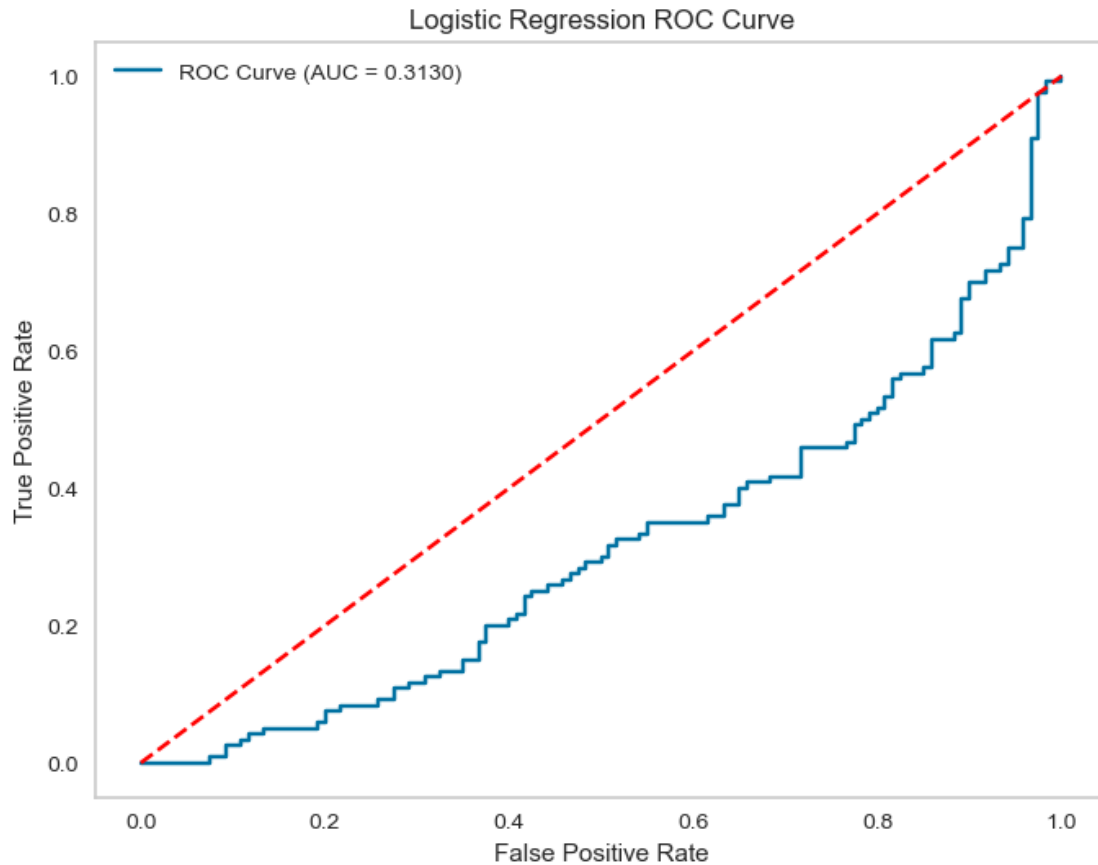
Precision: 0.4417

Recall: 0.7583

Roc_auc: 0.3130







0.6 Gradient Boosting (XGBoost)

```
[22]: # Step 1: Initialize the Gradient Boosting model
from xgboost import XGBClassifier

xgb = XGBClassifier(objective='binary:logistic', eval_metric='logloss',
    ↪ use_label_encoder=False,
    scale_pos_weight=1.0, random_state=42) # Handle imbalanced
    ↪ data via scale_pos_weight

# Step 2: Train the Gradient Boosting model on the training data
xgb.fit(X_train, y_train)

# Step 3: Predict on the validation set
y_pred_xgb = xgb.predict(X_scaled_val)
y_pred_proba_xgb = xgb.predict_proba(X_scaled_val)[: , 1] # Probability of
    ↪ class 1

# Step 4: Evaluate the Gradient Boosting model using validation data
```

```

accuracy_xgb = accuracy_score(y_validation_encoded, y_pred_xgb)
precision_xgb = precision_score(y_validation_encoded, y_pred_xgb)
recall_xgb = recall_score(y_validation_encoded, y_pred_xgb)
roc_auc_xgb = roc_auc_score(y_validation_encoded, y_pred_proba_xgb)

# Step 5: Visualize and summarize performance
print("Gradient Boosting (XGBoost) Evaluation:")
xgb_metrics = evaluate_model(xgb, X_scaled_val, y_validation_encoded,
                             label_encoder, model_name="XGBoost")
joblib.dump(xgb, 'trained_model_XGB_CSP.pkl')

```

Gradient Boosting (XGBoost) Evaluation:

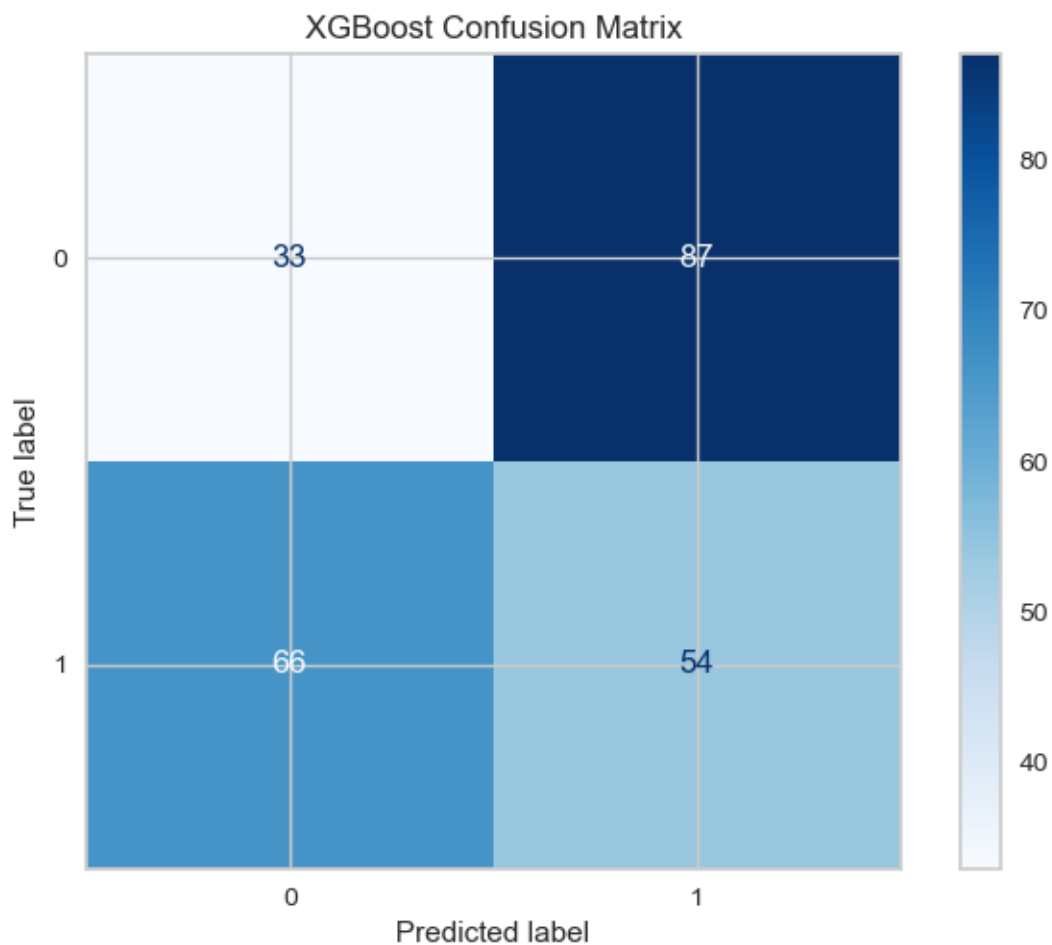
XGBoost Evaluation Metrics:

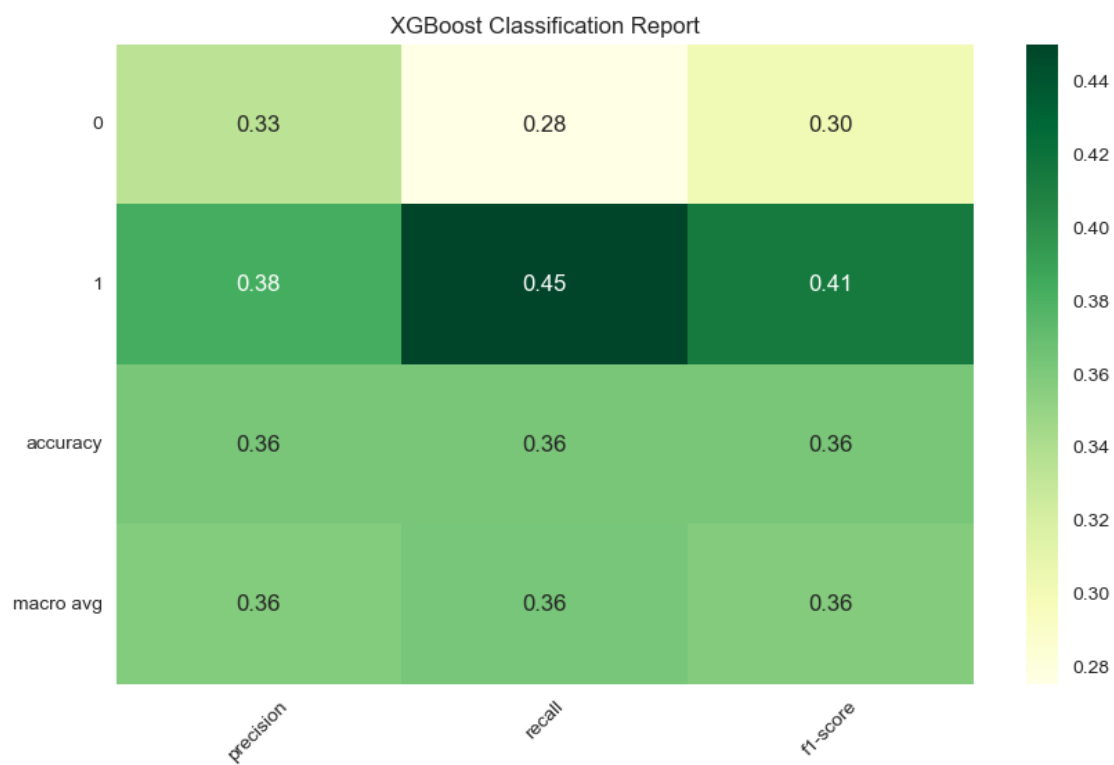
Accuracy: 0.3625

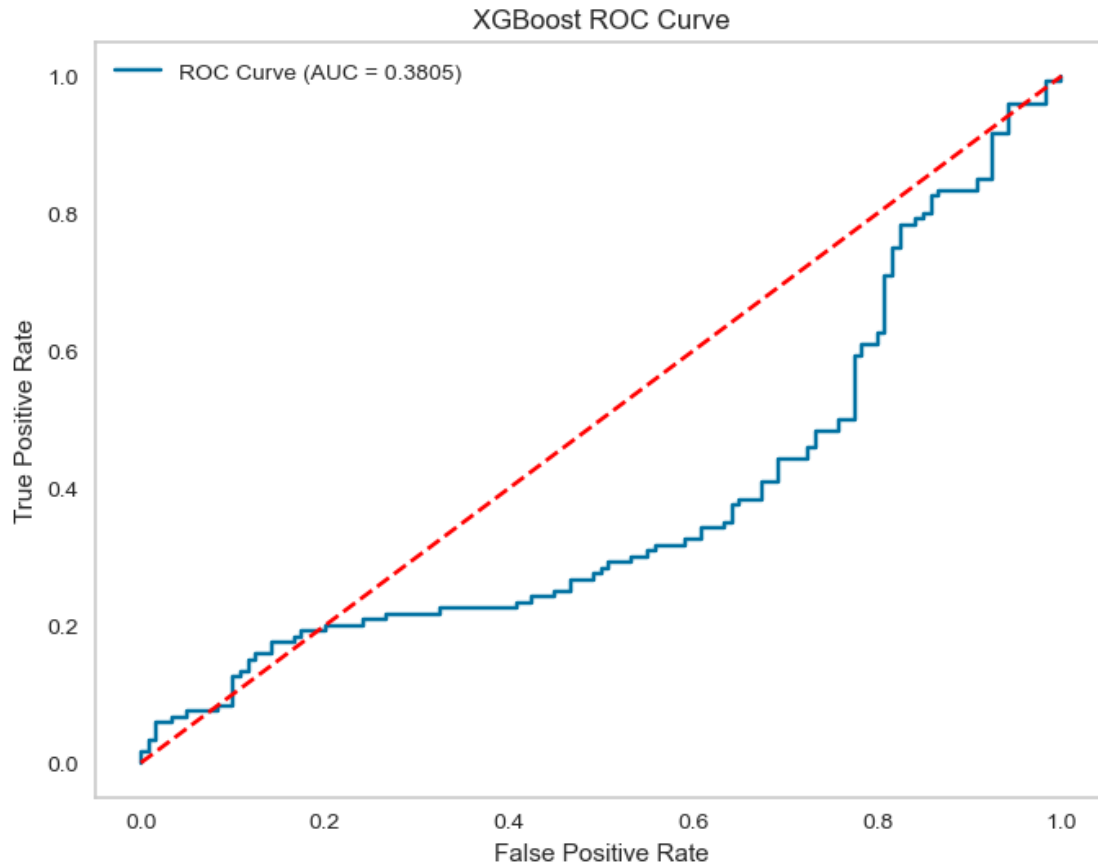
Precision: 0.3830

Recall: 0.4500

Roc_auc: 0.3805







```
[22]: ['trained_model_XGB_CSP.pkl']
```

```
[23]: import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

def compare_models(metrics_dict, roc_curves, labels):
    """
    Generates improved comparative plots and a metrics table for classification
    models.

    Parameters:
    - metrics_dict: A dictionary with model names as keys and metric
    dictionaries as values.
    - roc_curves: A dictionary with model names as keys and (fpr, tpr, roc_auc)
    as values.
    - labels: List of model names.
    """
    # Comparative Metrics
```

```

metrics_to_plot = ['accuracy', 'precision', 'recall', 'roc_auc']
n_metrics = len(metrics_to_plot)

fig, ax = plt.subplots(1, 2, figsize=(16, 8))

# Grouped bar plot for metrics
bar_width = 0.2
x = np.arange(len(labels))

for i, metric in enumerate(metrics_to_plot):
    values = [metrics_dict[label][metric] for label in labels]
    ax[0].bar(x + i * bar_width, values, width=bar_width, label=metric)

ax[0].set_title("Comparative Metrics", fontsize=16)
ax[0].set_ylabel("Score", fontsize=14)
ax[0].set_xlabel("Models", fontsize=14)
ax[0].set_xticks(x + bar_width * (n_metrics - 1) / 2)
ax[0].set_xticklabels(labels, rotation=45, fontsize=12)
ax[0].legend(fontsize=12)
ax[0].grid(axis='y', linestyle='--', alpha=0.7)

# ROC Curves
for label in labels:
    fpr, tpr, roc_auc = roc_curves[label]
    ax[1].plot(fpr, tpr, label=f"{label} (AUC={roc_auc:.2f})")

ax[1].plot([0, 1], [0, 1], linestyle='--', color='gray', alpha=0.7)
ax[1].set_title("ROC Curves", fontsize=16)
ax[1].set_xlabel("False Positive Rate", fontsize=14)
ax[1].set_ylabel("True Positive Rate", fontsize=14)
ax[1].legend(fontsize=12)
ax[1].grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()

# Create a table for numeric metrics
metrics_df = pd.DataFrame(metrics_dict).T[metrics_to_plot]
metrics_df.index.name = "Model"

print("\n### Numeric Comparison Table ###")
display(metrics_df) # If using Jupyter Notebook
# return metrics_df

```

```

labels = ["Decision Tree", "KNN", "Logistic Regression", "Random Forest"]

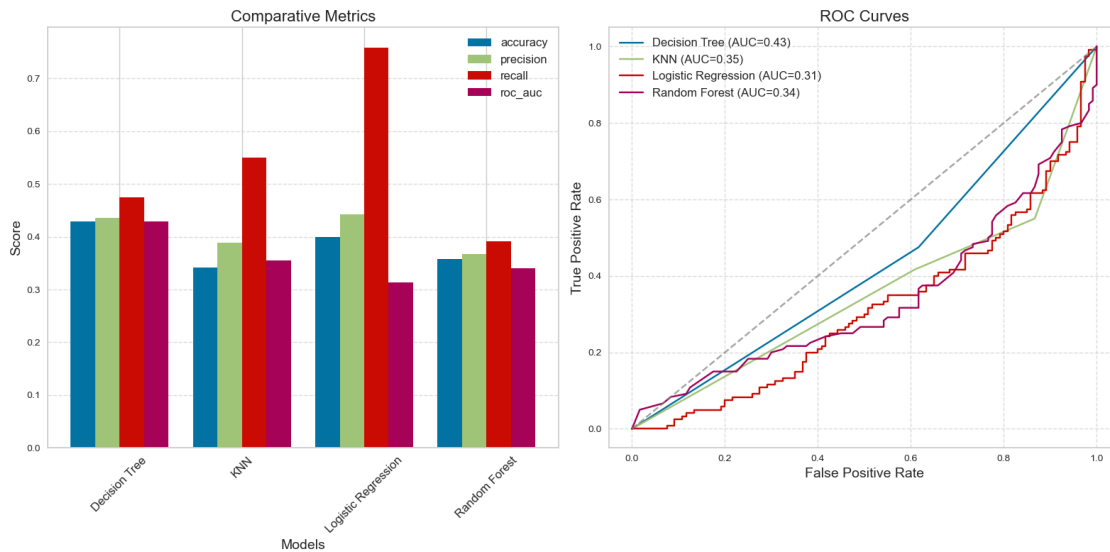
```

```

metrics_dict = {
    "Decision Tree": dt_metrics,
    "KNN": knn_metrics,
    "SVM": svm_metrics,
    "Logistic Regression": lr_metrics,
    "Random Forest": rf_metrics,
}
roc_curves = {
    "Decision Tree": (fpr_dt, tpr_dt, roc_auc_dt),
    "KNN": (fpr_knn, tpr_knn, roc_auc_knn),
    "SVM": (fpr_svm, tpr_svm, roc_auc_svm),
    "Logistic Regression": (fpr_lr, tpr_lr, roc_auc_lr),
    "Random Forest": (fpr_rf, tpr_rf, roc_auc_rf),
}

compare_models(metrics_dict, roc_curves, labels)

```



Numeric Comparison Table

Model	accuracy	precision	recall	roc_auc
Decision Tree	0.429167	0.435115	0.475	0.429167
KNN	0.341667	0.388235	0.55	0.354722
SVM	0.395833	0.439614	0.758333	0.291007
Logistic Regression	0.4	0.441748	0.758333	0.312986
Random Forest	0.358333	0.367188	0.391667	0.34066

[1] Khan J, Bhatti MH, Khan UG, Iqbal R. Multiclass EEG motor-imagery classification with sub-band common spatial patterns. EURASIP J Wirel Commun Netw [Internet]. 2019;2019(1).

Available from: <http://dx.doi.org/10.1186/s13638-019-1497-y>

[2] Saibene A, Caglioni M, Corchs S, Gasparini F. EEG-based BCIs on motor imagery paradigm using wearable technologies: A systematic review. *Sensors (Basel)* [Internet]. 2023;23(5):2798. Available from: <https://www.mdpi.com/1424-8220/23/5/2798>

[3] Wang Y, Gao S, Gao X. Common spatial pattern method for channel selection in motor imagery based brain-computer interface. In: 2005 IEEE Engineering in Medicine and Biology 27th Annual Conference. IEEE; 2005. p. 5392–5.

[4] Omari S, Omari A, Abderrahim M. Multiple tangent space projection for motor imagery EEG classification. *Appl Intell* [Internet]. 2023;53(18):21192–200. Available from: <http://dx.doi.org/10.1007/s10489-023-04551-2>

[5] Casso M-I, Jeunet C, Roy RN. Heading for motor imagery brain-computer interfaces (MI-BCIs) usable out-of-the-lab: Impact of dry electrode setup on classification accuracy. In: 2021 10th International IEEE/EMBS Conference on Neural Engineering (NER). IEEE; 2021. p. 690–3.